

Setup guide

RAG SaaS Platform

SYSTEM_REQUIREMENTS

System Requirements

Self-hosted (Docker Compose)

- Docker Engine + Docker Compose v2.
- A reasonable baseline for small deployments: 4 vCPU / 8 GB RAM for

the combined stack (`api`, `celery-worker`, `frontend`, `postgres`, `redis`). Increase worker/API replicas and resources with usage — see [Scaling](#).

- Additional CPU (or GPU, if configured) headroom if you enable local

vision inference (YOLOv8 object detection, CLIP embedding) — these run local model inference rather than calling an external API.

Database

An external managed Postgres (e.g. Supabase) with `pgvector` enabled is recommended over the bundled `postgres` container for anything beyond small/trial deployments — see [Database Setup](#).

Object storage

An S3-compatible bucket for documents, media, and fine-tuning datasets.

Network

Outbound HTTPS access is required for LLM provider calls (via litellm), transactional email (Resend), and any configured integrations (Slack/Teams/Discord/Twilio/Airbyte/n8n).

Browser support (frontend)

A current Chromium-, Firefox-, or Safari-based browser. Voice features specifically work most consistently on Chromium-based browsers — see [Voice](#).

SELF_HOSTED

Self-hosted

Full detail on the self-hosted sales/deployment model is in [docs/sales/SELF_HOSTED.md](#). This page covers the actual installation steps.

Requirements

Docker Engine + Docker Compose v2. See [System Requirements](#) for resource sizing.

Install

```
./install.sh
```

First run copies `.env.example` to `.env`, generates a real random `SESSION_MIDDLEWARE_SECRET`, and stops — asking you to fill in `DATABASE_URL`, `RESEND_API_KEY`, and a license key. See [Environment Variables](#) for the full list.

Second run (after `.env` is filled in) starts the stack via `docker-compose.selfhosted.yml`, waits for Postgres to become healthy, and runs Alembic migrations automatically.

Upgrading

```
./update.sh
```

Takes a real backup first (`scripts/backup.sh`), pulls the latest code, rebuilds, and migrates — with a rollback path documented on failure. See [Upgrading](#) and [Backup & Restore](#).

Uninstalling

```
./uninstall.sh           # stops and removes containers/network only
./uninstall.sh --purge-data # also deletes the Postgres volume and backups
```

`--purge-data` is opt-in and explicit — a plain uninstall never deletes your `.env`, backups, or database volume. See [Uninstall](#).

Next steps

[Database Setup](#), [SSL & Domains](#), [Observability Stack](#).

SSL_AND_DOMAINS

SSL & Domains

For custom domains and white-label branding, see [docs/whitelabel/DOMAIN.md](#) and [White-label](#). This page covers the SSL mechanics specifically.

How SSL is provisioned

SSL certificates for custom domains are provisioned via a real ACME client using the DNS-01 challenge, verified against the Let's Encrypt staging environment. The private key/certificate material is encrypted at rest using the platform's shared encryption-at-rest key (`api/security/encryption.py`) — see `api/routers/ssl_certificates.py`.

Honest limitation: not fully automatic

DNS-01 requires a DNS TXT record to be created for the challenge. This platform does not currently integrate with DNS provider APIs (Cloudflare, Route53, etc.) to create that record automatically — you create the TXT record manually (or via your own DNS automation) when prompted, then trigger verification. This is a real, documented gap, not a bug: full automation would need a DNS-provider integration that doesn't exist yet.

Domain verification

Once a domain is added, `api/routers/custom_domains.py` polls periodically (Celery) to confirm the domain still points at your deployment and the certificate remains valid, renewing before expiration.

Reverse proxy

Serving traffic for an active custom domain by `Host` header requires a reverse proxy configured to route accordingly — this is part of your own infrastructure setup, not automated by the platform itself.

UPGRADING

Upgrading

Self-hosted

```
./update.sh
```

What it actually does, in order:

1. Takes a real backup via `scripts/backup.sh` first — the update

script refuses to skip this step.

1. `git pull --ff-only`.
2. Rebuilds and restarts the stack.
3. Runs Alembic migrations.
4. On any failure past the backup step, tells you exactly which backup

file to restore from (`./scripts/restore.sh <backup_file>`) rather than leaving you to figure it out.

See [Backup & Restore](#) for the restore path.

Before upgrading

- Read the relevant entries in [CHANGELOG.md](#) for

anything that needs manual attention (new required environment variables, breaking config changes).

- Confirm you have enough disk space for a new backup.

SaaS

Managed upgrades are handled for you — no action required. See [SaaS](#).

SCALING

Scaling

Stateless services scale horizontally

`api` and `frontend` are stateless — run multiple replicas behind a load balancer as traffic requires. `celery-worker` also scales horizontally; add more workers to increase background-job throughput (document ingestion, evaluation runs, media processing, fine-tuning polling).

Do not scale `celery-beat`

`celery-beat` (the scheduler for periodic tasks — reindexing, domain verification, fine-tuning status polling, backups) must run as exactly one instance. Running more than one causes duplicate scheduled task execution.

Database

Postgres is the primary scaling constraint for most deployments. Use a managed Postgres (e.g. Supabase) with connection pooling (PgBouncer or equivalent) before scaling `api` replicas significantly — async SQLAlchemy connections multiply quickly with concurrent replicas.

Redis

Used as the Celery broker/result backend. A single Redis instance is sufficient for most deployments; consider Redis Cluster or a managed Redis only at high background-job volume.

Vision/media workloads

YOLO object detection and CLIP embedding run local inference and are CPU/GPU-bound — if media volume is high, dedicate `celery-worker` capacity (or a separate worker pool bound to the media queue) rather than letting media processing compete with lighter background jobs.

TROUBLESHOOTING

Troubleshooting

`install.sh` exits immediately after creating `.env`

Expected on first run — it stops so you can fill in `DATABASE_URL`, `RESEND_API_KEY`, and a license key before continuing. Re-run `./install.sh` once `.env` is filled in. See [Environment Variables](#).

Migrations fail on startup

Check `DATABASE_URL` uses the `asyncpg` driver (`postgresql+asyncpg://...`), not plain `postgresql://` — a common copy-paste mistake from a generic Postgres connection string. See [Database Setup](#).

Frontend can't reach the API (CORS errors)

The frontend's CORS origin variable in `.env` must match the port your frontend actually runs on. See the "Frontend / CORS" section of [Environment Variables](#).

Fine-tuning jobs fail immediately for Anthropic

Expected — Anthropic has no public fine-tuning API. Use OpenAI or Mistral instead. See [docs/fine-tuning/OVERVIEW.md](#).

Custom domain SSL stuck "pending"

The DNS-01 TXT record likely hasn't propagated yet, or wasn't created correctly. See [SSL & Domains](#) — this step is manual, not automatic.

`celery-beat` running scheduled tasks twice

You likely have more than one `celery-beat` instance running. It must run as exactly one instance — see [Scaling](#).

Still stuck

Check `docker compose -f docker-compose.selfhosted.yml logs -f api` for the actual error, and see [Observability Stack](#) if you have monitoring configured.

UNINSTALL

Uninstall

```
./uninstall.sh
```

Stops and removes the containers and network `docker-compose.selfhosted.yml` created. It deliberately does **not** touch `.env`, your `backups/` directory, or Postgres's named data volume — an uninstall that silently deletes real data is the one mistake this script is written to avoid.

Removing data too

```
./uninstall.sh --purge-data
```

Only with this explicit flag does uninstall also remove the Postgres volume and backups. There is no prompt-based confirmation — the flag itself is the confirmation, so don't pass it unless you mean it.

Before purging

If you might reinstall later, take a fresh backup first regardless of whether `--purge-data` already implies backups will be removed:

```
./scripts/backup.sh
```

See [Backup & Restore](#).